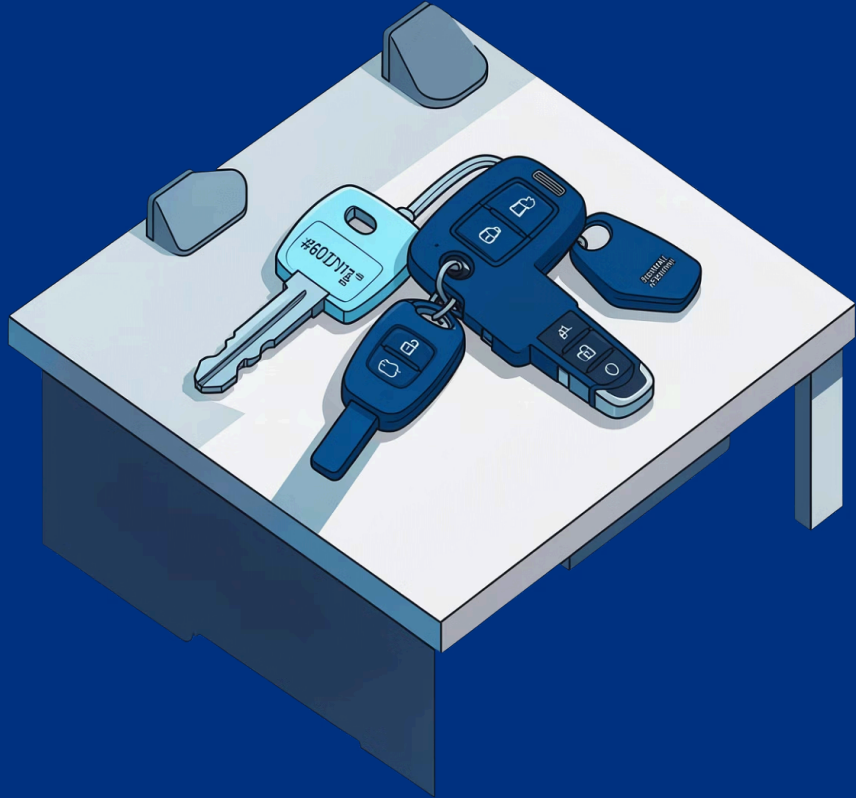




Car Rental Fleet Manager

Programming for Problem Solving (PSPJ) – Java |

PRESENTED BY: Gundu Meghana | 2620030053

Abstract

The **Car Rental Fleet Manager** is a console-based Java application that manages a small fleet of rental cars, demonstrating fundamental programming concepts through a practical, real-world problem-solving approach.

Core Functionality

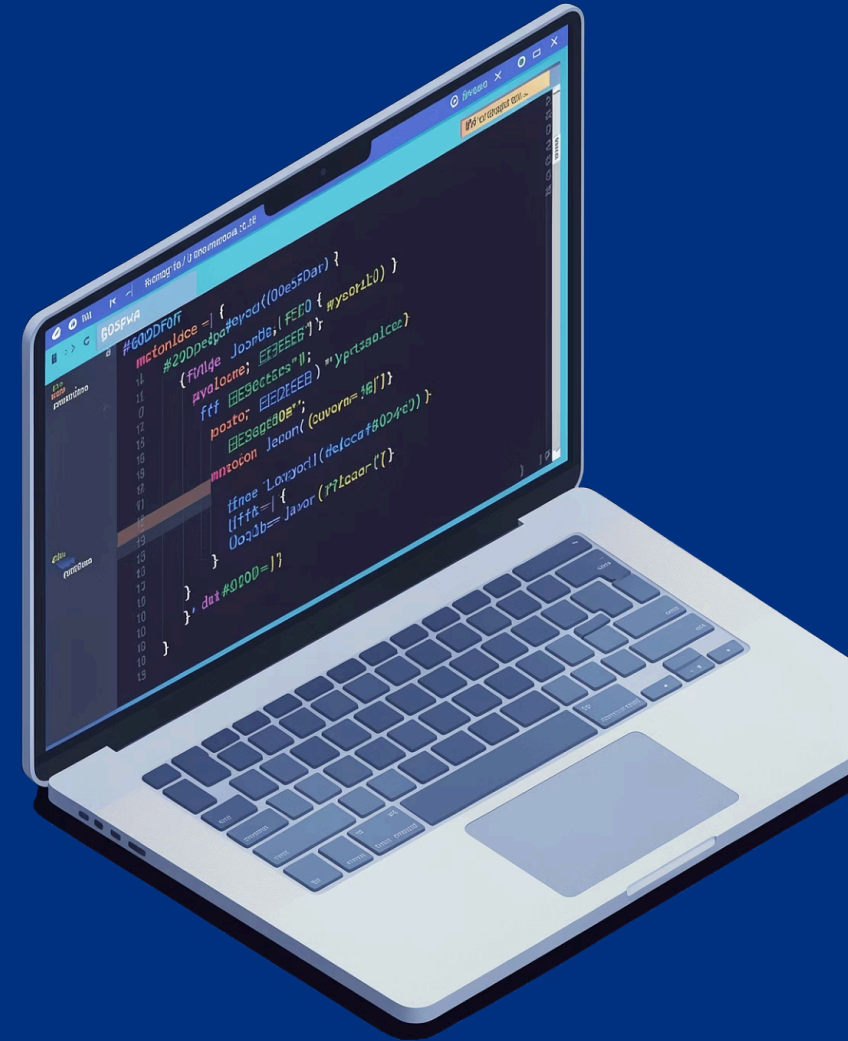
View cars, search vehicles, check availability, rent/return cars, calculate charges, sort by price

Technology Stack

Pure Java — arrays, methods, and control structures only.
No advanced APIs.

PSP Concepts

Modules 1, 2, and 3 integrated into one cohesive, compilable application





Problem Identification & Objectives

Problem Statement

Manual fleet management makes it difficult to track car availability, rental prices, customer selections, and rental duration efficiently. A structured programmatic solution is required.

Input → Process → Output Model

User commands feed into control-flow logic, which processes fleet arrays and delivers formatted console output — a direct demonstration of the IPO model from Module 1.

Project Objectives

- 1 Maintain car information**
ID, model, price per day, availability status
- 2 Search & display cars**
Filter by ID or name using overloaded methods
- 3 Manage rent & return**
Update availability and compute charges dynamically
- 4 Sort fleet by price**
Bubble sort across parallel arrays using nested loops

Advantages of the Project

Real-world problem solving

Demonstrates practical application through a car rental management scenario.

Integrated learning

Combines key programming concepts from Modules 1–3 in one cohesive system.

Scalable design

Uses arrays and modular methods to support clean expansion and maintenance.

Easy to understand

Simple structure makes the project approachable for learning and modification.

Pure Java implementation

No external dependencies — built entirely with core Java features.

Core concepts covered

Covers control flow, data structures, sorting, and modular programming.

Module-Wise Concept Mapping

1

Java Fundamentals

- Computational thinking, algorithm design, flowchart
- Variables, primitive data types, arithmetic operators
- Scanner input, formatted console output with printf
- Ternary operator, type conversions

2

Control Flow

- if-else, switch, nested conditionals
- for, while, do-while loops
- break and continue statements
- Bubble sort using nested for loops

3

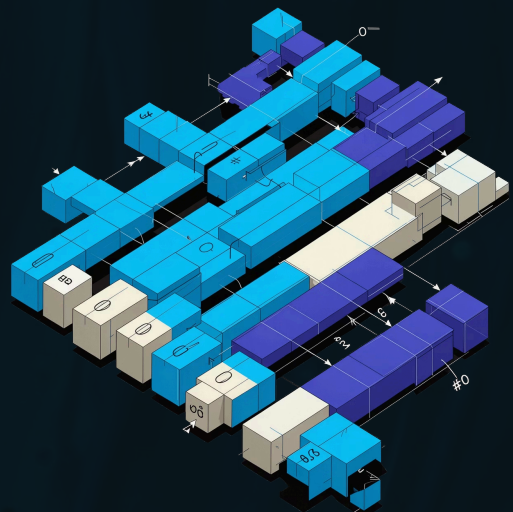
Modular Programming

- Methods with parameters and return values
- Method overloading (searchCar by ID & by name)
- Static arrays, enhanced for loop
- Linear search and bubble sort algorithms

System Design & Data Structures

Fleet Storage — Parallel Arrays

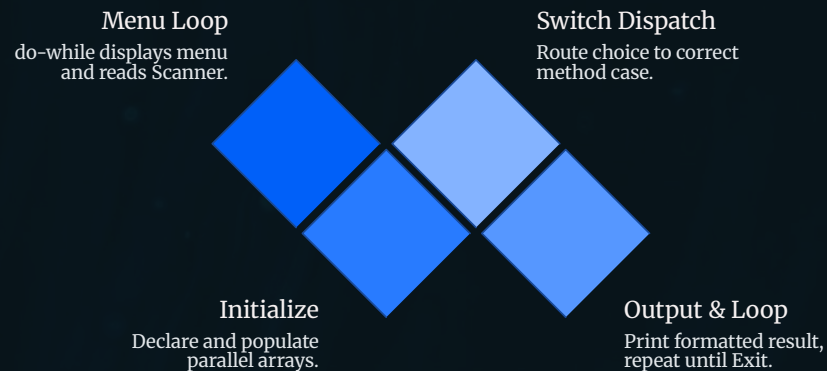
All fleet data is stored across five parallel static arrays. Each index *i* represents the same car across every array.



Array	Values
carId[]	101, 102, 103, 104, 105
carName[]	Swift, Baleno, Creta, City, Nexon
pricePerDay[]	1500, 1700, 2500, 2200, 1800
available[]	true, true, false, true, true
rentalDays[]	0, 0, 0, 0, 0 (tracks duration)

❏ Array index correspondence ensures data integrity across all operations — a core Module 3 design principle.

Algorithm & Program Flow



Main Algorithm Steps

01

Initialize fleet arrays

Static parallel arrays populated at class level

02

Display menu & read choice

do-while loop with Scanner — Module 2

03

Switch dispatches to method

8 cases — one per operation — Module 2

04

Method executes & returns result

Formatted printf output — Modules 1 & 3

05

Loop or exit

Repeats until choice == 8

Implementation: Key Methods

`void displayCars()`

Enhanced for loop iterates all array indices. Uses `printf` for tabular, formatted output. **Module 3 + Module 1.**

`int searchCar(int id) & searchCar(String name)`

Method overloading — same method name, different parameter types. Linear search returns index or -1. **Module 3.**

`void rentCar(sc) / returnCar(sc)`

if-else chain validates availability, updates boolean array, calls `calculateRent()`. **Module 2 + Module 3.**

`int calculateRent(int days, int price)`

Returns `days * pricePerDay` — demonstrates method with parameters and arithmetic return value. **Module 1 + Module 3.**

`void sortCarsByPrice()`

Bubble sort with **nested for loops** across temporary parallel arrays. Swaps all four arrays in sync. **Module 2 + Module 3.**

`void displaySummary()`

Counting loop accumulates `availableCount`, `rentedCount`, `totalRevenue`. Ternary operator for status. **Module 1 + Module 2.**

Complete Java Implementation

```
import java.util.Scanner;

public class CarRentalFleetManager {

    // MODULE 1: Variables & Data Types
    static int[] carId = {101, 102, 103, 104, 105};

    static String[] carName = {
        "Swift", "Baleno", "Creta", "City", "Nexon"
    };

    static int[] pricePerDay = {
        1500, 1700, 2500, 2200, 1800
    };

    static boolean[] available = {
        true, true, false, true, true
    };

    static int[] rentalDays = {
        0, 0, 0, 0, 0
    };

    static int fleetSize = 5;

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);
        int choice;

        do {
            // MODULE 2: do-while loop
            displayMenu();

            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            // MODULE 2: switch statement
            switch (choice) {

                case 1:
                    displayCars();
                    break;

                case 2:
                    searchCarMenu(sc);
                    break;

                case 3:
                    rentCar(sc);
                    break;

                case 4:
                    returnCar(sc);
                    break;

            }

        } while (choice != 0);
    }
}
```

Complete Java Implementation – Part 2

```
// MODULE 3: Method with parameter
static void searchCarMenu(Scanner sc) {
    System.out.println("\nSearch Car");
    System.out.println("1. Search by ID");
    System.out.println("2. Search by Name");
    System.out.print("Enter your choice: ");
    int choice = sc.nextInt();
    sc.nextLine();

    int index = -1;

    if (choice == 1) {
        System.out.print("Enter car ID: ");
        int id = sc.nextInt();
        index = searchCar(id);
    } else if (choice == 2) {
        System.out.print("Enter car name: ");
        String name = sc.nextLine();
        index = searchCar(name);
    } else {
        System.out.println("Invalid search option!");
        return;
    }

    if (index != -1) {
        System.out.println("Car found:");
        System.out.println("ID: " + carId[index]);
        System.out.println("Model: " + carName[index]);
        System.out.println("Price/Day: Rs. " + pricePerDay[index]);
        System.out.println("Status: " + (available[index] ? "Available" : "Rented"));
    } else {
        System.out.println("Car not found!");
    }
}
```

Sample Console Output

```
PS C:\Users\MEGHANA G> & 'C:\Program Files\Eclipse Adoptium\jdk-21.0.12.8-hotspot\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\MEGHANA G\AppData\Local\Temp\
vscodesws_8b954\jdt_ws\jdt.ls-java-project\bin' 'CarRentalFleetManager'
1. Display Cars      5. Calculate Rent
2. Search Car        6. Sort by Price
3. Rent Car          7. Fleet Summary
4. Return Car        8. Exit
=====
Enter your choice: 4
Enter Car ID to return: 3
Car not found!

===== CAR RENTAL FLEET MANAGER =====
1. Display Cars      5. Calculate Rent
2. Search Car        6. Sort by Price
3. Rent Car          7. Fleet Summary
4. Return Car        8. Exit
=====
```

Sample Console Output

```
=====
Enter your choice: 3
Enter Car ID to rent: 102
Rental days: 8
Rented: Baleno | Days: 8 | Total: Rs.13600
```

```
===== CAR RENTAL FLEET MANAGER =====
1. Display Cars      5. Calculate Rent
2. Search Car        6. Sort by Price
3. Rent Car          7. Fleet Summary
4. Return Car        8. Exit
```

Worked Example & Logic Walkthrough

Scenario: Rent a Hyundai Creta

Car Selected

Creta — ID: 103 | Price: ₹2500/day

Duration

3 rental days entered by user

Calculation (Module 1)

`calculateRent(3, 2500) →`
 $2500 \times 3 = ₹7500$

Unavailability Guard (Module 2)

If `available[index] == false`,
if-else block prevents rental
and prints error message

❑ Modules demonstrated: Module 1 (arithmetic operators), Module 2 (if-else validation), Module 3 (method with parameters & return value)

Bubble Sort Trace — `sortCarsByPrice()`

Pass	Sorted Order	Price
Original	Swift, Baleno, Creta, City, Nexon	1500 1700 2500 2200 1800
Pass 1	Swift, Baleno, Nexon, City, Creta	1500 1700 1800 2200 2500
Final	Swift, Baleno, Nexon, City, Creta	1500 1700 1800 2200 2500 ✓

Nested `for` loops compare adjacent prices and swap all four parallel arrays simultaneously to maintain data integrity.

Conclusion

The Car Rental Fleet Manager successfully demonstrates how fundamental programming concepts from Modules 1, 2, and 3 integrate into a practical, real-world application. Through this project, we've shown:

- Problem decomposition into manageable, reusable methods
- Effective use of data structures (arrays) for managing complex information
- Control flow logic (if-else, switch, loops) for decision-making and iteration
- Algorithm implementation (linear search, bubble sort) for data manipulation
- User input validation and error handling for robust applications

This project bridges the gap between theoretical programming concepts and practical problem-solving, equipping first-year CSE students with the foundational skills needed for advanced coursework in object-oriented programming, data structures, and software engineering.

Future Work & Advanced Extensions

The same project can evolve naturally into the next modules by strengthening structure, persistence, and scalability. The following extensions build directly on the existing fleet manager:

Module 4 Extensions

- Implement Car and Customer as classes with encapsulation
- Use constructors and method overloading for flexible object creation
- Add exception handling for invalid inputs and edge cases

Module 5 Extensions

- Persist fleet data to files using File I/O
- Parse CSV records for bulk car import/export
- Use StringBuilder for dynamic report generation

Module 6 Extensions

- Replace arrays with ArrayList and HashMap for dynamic fleet management
- Implement Comparable interface for flexible sorting criteria
- Use Stream API for advanced filtering and aggregation
- Apply lambda expressions for custom search and sort operations

Thank You

Thank you for reviewing the Car Rental Fleet Manager project.

GitHub Repository: [Project Link](#)

"Programming is not about writing code—it's about solving problems systematically."

